



University of Asia Pacific

Department of Computer Science and Engineering

CSE 316: Microprocessors and Microcontrollers Lab

LAB REPORT

Experiment Number: 01

Experiment Title: Familiarization with the MDA-8086 Microprocessor Trainer and EMU8086 Microprocessor Emulator.

Submitted by:

Name : Sunandan Das

Student ID : 23101218

Section : D2

Submitted to:

Zaima Sartaj Taheri

Lecturer,

Department of Computer Science and Engineering

Date of Submission: 06/02/2026

1. Experiment Name

Understand the component of the 8086 trainer board.

2. Objective

- Understand the component of 8086 trainer board.
- Understand the EMU8086 Microprocessor Emulator.
- Learn 8086 16-bit Intel Microprocessor, its register and assembly level programming.
- Learn assembly programming by practicing simple programs including average calculation of 3/4/5/more numbers, calculate area of a rectangle and a triangle, temperature conversion from °C to °F, conversion from °F to °C, conversion from °C to °K, conversion from °K to °C and counting tiles problems.

3. Circuit Diagram / Schematic

-MDA-8086 Kit Diagram



Figure 1.1: MDA-8086 Kit Diagram

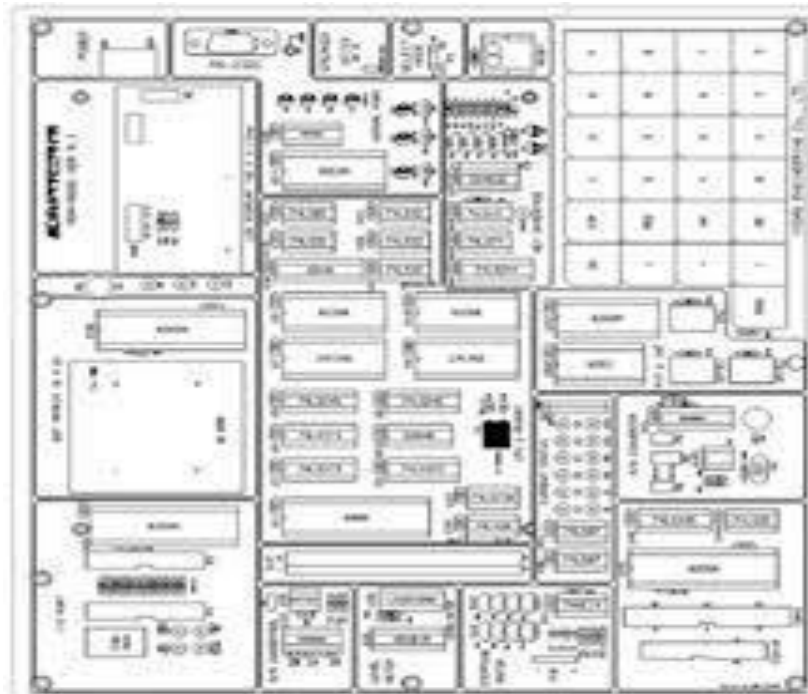


Figure 1.2: MDA-8086 Kit Circuit Diagram

The function of IC's at MDA-8086 System Configuration

1. CPU (Central processing unit): Using Intel 8086, using 14.7456MHz.
2. ROM (Read Only Memory): It has program to control user's key input, LCD display, user's program. 64K Byte, it has data communication program. Range of ROM Address is F0000H FFFFFH.
3. SRAM (Static Random-Access Memory): Input user's program & data. Address of memory is 00000H 0FFFFH, totally 64K Byte.
4. DISPLAY: Text LCD Module, 16(Characters)×2(Lines)
5. KEYBOARD: It is used to input machine language. There are 16 hexadecimal keys and 8 function keys.
6. SPEAKER: Sound test.
7. RS-232C: Serial communication with IBM compatible PC.
8. DOT MATRIX LED: To understand & test the dot matrix structure and principle of display. It is interfaced to 8255A(PPI).
9. A/D CONVERTER: ADC0804 to convert the analog signal to a digital signal.

10. D/A CONVERTER: DAC0800 (8-bit D/A converter) to convert the digital signal to the analog signal and to control the level meter.
11. STEPPING MOTOR INTERFACE: Stepping motor driver circuit is designed.
12. POWER: AC 110~220V, DC +5V 3A, +12V 1A, -12V 0.5A SMPS.

Operation Introduction

MDA-8086 has high performance 64K-byte monitor program. It is designed for easy function. After power is on, the monitor program begins to work. In addition to all the key function the monitor has a memory checking routine.

FUNCTION KEY				DATA KEY	
				MON	RES
GO	STP	C	D	E	F
+	REG	8	9	A	B
-	DA	4	5	6	7
:	AD	0	1	2	3

- RES→ System reset
- STP→ Execute user's program, a single step
- AD→ Set memory address
- GO→ Go to user's program or execute monitor functions
- DA→ Update segment & Offset and input data to memory
- MON→ Immediately break user's program and Non maskable interrupt.
- : → Offset set
- REG→ Register Display.
- +→ Segment & Offset +1 increment. Register display increment.
- -→ Segment & Offset -1 increment. Register display decrement.

8255 Programmable Peripheral Interface Controller

- It has 24-bit input/output pins
- It consists of three ports: port A, port B and port C- all of which are 8 bits
- It also consists of an 8-bit control register (CR)
- The eight bit of port C can be used as individual bits or be grouped in two 4-bit ports:

Cupper(CU) and C lower(CL)

- The functions of these ports are defined by writing a control word in the control Register

Group A	Group B
Port A	Port B
Port C (Upper 4 bit)	Port C (Lower 4 bit)

8086 Instruction Set Summary

Data Registers	AX (Accumulator Register)	AH	AL
	BX (Base Register)	BH	BL
	CX (Count Register)	CH	CL
	DX (Data Register)	DH	DL
Segment Registers	CS (Code Segment)		
	DS (Data Segment)		
	SS (Stack Segment)		
	ES (Extra Segment)		
Index Registers	SI (Source Index)		
	DI (Destination Index)		
Pointer Registers	SP (Stack Pointer)		
	BP (Base Pointer)		
	IP (Instruction Pointer)		
	FLAGS Registers		

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
				OF	DF	IF	TF	SF	ZF		AF		PF		CF

Bit	Name	Symbol	Status Flags
0	Carry Flag	CF	
2	Parity Flag	PF	
4	Auxiliary Carry Flag	AF	
6	Zero Flag	ZF	
7	Sign Flag	SF	
11	Overflow Flag	OF	Control Flags
8	Trap Flag	TF	
9	Interrupt Flag	IF	
10	Direction Flag	DF	

Data Transfer Instructions

Name	Mnemonic
Load	LD
Store	ST
Move	MOV
Exchange	XCHG
Input	IN
Output	OUT
Push	PUSH
Pop	POP

Arithmetic

Name	Mnemonic
Increment	INC
Decrement	DEC
Add	ADD
Subtract	SUB
Multiply	MUL
Divide	DIV
Add with carry	ADDC
Subtract with borrow	SUBB
Negate	NEG

Logical and Bit Manipulation

Name	Mnemonic
Clear	CLR
Complement	COM
AND	AND
OR	OR
Exclusive-OR	XOR
Clear carry	CLRC
Set carry	SETC
Complement carry	COMC
Disable interrupt	DI

Shift and Rotate

Name	Mnemonic
Logical shift right	SHR
Logical shift left	SHL
Arithmetic shift right	SHRA
Arithmetic shift left	SHLA
Rotate right	ROR
Rotate left	ROL
Rotate right through carry	RORC
Rotate left through carry	ROLC

Program Control Instructions

Name	Mnemonic
Branch	BR
Jump	JMP
Skip	SKP
Call	CALL
Return	RET
Compare (Subtract)	CMP
Test (AND)	TST

4. Conclusion

In this experiment, we gained hands-on experience with the MDA-8086 trainer board and the EMU8086 emulator by studying the main hardware sections and observing the system's behavior immediately after power-up. We explored the functions of the 8086 microprocessor, the ROM and SRAM memory organization, and the built-in interfaces including the LCD, hexadecimal keypad, RS232 serial interface, and peripheral units such as the dot-matrix LED controlled via the 8255 PPI, ADC0804, DAC0800, and the stepper motor module. Additionally, we used the trainer's monitor commands—such as system reset, memory address and data input, register inspection, single-step execution, and program execution with breakpoints—to monitor changes in memory and registers. Overall, this experiment provided a solid base for developing and debugging simple 8086 assembly programs and for understanding the interaction between software instructions and actual hardware and I/O operations.